Problem Set 5

Due February 20, 2026 at 11:59 PM Pacific

Turn in your problem set by Gradescope. Include any code you write in your solutions as well. Mark down how much time you spent on the homework set.

1. [15 points] **State and Parameter Estimation:** The EKF can be used to estimate unknown parameters of a dynamical system, while also estimating its state. Consider the roll mode of a fixed-wing aircraft, with the model $\dot{p} = -ap + bu$, where p is roll rate, u is an aileron input, and a, b are unknown (fixed) model parameters. You receive a measurement $y(t) = p(t)$ from the IMU.

   (a) Formulate a discrete time approximation of this system $x_{t+1} = f(x_t, u_t) + w_t$, $y_t = g(x_t) + v_t$ with the state $x = [p \quad a \quad b]^T$.

   (b) Find the Jacobians A_t and C_t for the EKF.

   (c) Simulate the system with true parameters $a = 0.5$, $b = 2$, with an input $u_t = \sin(t)$, with $p(0) = 5$, and with additive process noise covariance $Q = \text{diag}([0.1 \quad 0 \quad 0])$ and measurement noise covariance $R = 0.1$. Implement an EKF to estimate the parameters a and b, as well as p. Use the noise covariances $Q = \text{diag}([0.1 \quad 0.01 \quad 0.01])$, $R = 0.1$ in your EKF. Start with $\Sigma_0 = 10I_3$ and $\mu_0 = 0$. Use $dt = 0.1s$, and simulate for 100 seconds.
   Remark: Think about what this result represents. You only have a model of your system, not an exact specification. But this method lets you estimate from measurements, in real-time, not just your real state but also the parameters of the system!

2. [15 points] **Kalman Filter as an Optimization Problem**: In class we claimed that the Kalman Filter update equations can be derived as the analytical solution to a least squares optimization problem. Let's prove the claim. Consider the Maximum A Posteriori (MAP) optimization problem we saw in class, $x_t^{\text{MAP}} = \arg\min_{x_t} J(x_t)$. With a linear measurement model $Y_t = C_t X_t + V_t$, $V_t \sim \mathcal{N}(0, R_t)$, and Gaussian prediction distribution $p(x_t \mid y_{1:t-1}) = \mathcal{N}(\mu_{t|t-1}, \Sigma_{t|t-1})$, we saw that the MAP objective function becomes a convex quadratic in x_t,

$$J(x_t) = \frac{1}{2}\|y_t - C_t x_t\|^2_{R_t^{-1}} + \frac{1}{2}\|x_t - \mu_{t|t-1}\|^2_{\Sigma^{-1}_{t|t-1}}.$$

   (a) Analytically find x_t^{MAP} by setting the gradient of the objective function to zero $\frac{\partial J(x_t)}{\partial x_t}\,|_{x_t = x_t^{\text{MAP}}} = 0$ and solving for x_t^{MAP}. Verify that $\mu_{t|t} = x_t^{\text{MAP}}$ by showing that your expression matches the mean update equation from the Kalman filter.

   (b) Analytically find the Hessian, $H = \frac{\partial^2 J(x_t)}{\partial x_t^2}\,|_{x_t = x_t^{\text{MAP}}}$. Show that $\Sigma_{t|t} = H^{-1}$ by verifying that the inverse of your expression for H matches the covariance update equation from the Kalman Filter.

   Hint: the matrix inverse lemmas that we have seen in previous problem sets will come in handy for this problem!

3. [20 points] **EKF SLAM**: Consider again the differential drive robot model,

$$\begin{aligned} P^x_{t+1} &= P^x_t + \delta t\, s_t \cos\Theta_t + W^x_t \\ P^y_{t+1} &= P^y_t + \delta t\, s_t \sin\Theta_t + W^y_t \\ \Theta_{t+1} &= \Theta_t + \delta t\, \phi_t + W^\theta_t \end{aligned}$$

   where $P_t = [P^x_t \quad P^y_t]^T$ is the robot position relative to a feature point, Θ_t is its heading angle, s_t its speed input, ϕ_t its rotation rate input, and W_t is the process noise distributed as $W_t \sim \mathcal{N}(0, Q)$ with $Q = 0.1\delta t I_3$. Suppose you have four features at positions $m_1 = [0 \quad 0]^T$, $m_2 = [10 \quad 0]^T$, $m_3 = [10 \quad 10]^T$, and $m_4 = [0 \quad 10]^T$. Now suppose the robot takes relative position measurements to each of the features (in the robot's local frame), with measurement model,

$$Y^j_t = \begin{bmatrix} \cos(\Theta_t) & \sin(\Theta_t) \\ -\sin(\Theta_t) & \cos(\Theta_t) \end{bmatrix} (m_j - P_t) + V^j_t,$$

as we saw in class. The measurements have an associated measurement uncertainty of $V_t^j \sim \mathcal{N}(0, R)$, where $R = 0.25I_2$. Also suppose that the robot pose is initially distributed $\mathcal{N}([1m\ 1m\ 0rad]^T, I_3)$ is commanded with a known control sequence $s_t = 0.5$ and $\phi_t = \sin(2t)$rad/s. You can assume that the ground truth trajectory also starts at $[1m\ 1m\ 0rad]^T$. Use $\delta t = 0.5s$ and run for 100 timesteps (i.e., $50s$).

We highly recommend using the starter code from Problem Set 4 Q4, and editing it as necessary for this problem.

(a) Calculate the measurement Jacobian C_t^b for the measurement model.

(b) Implement an EKF for the robot to localize itself with these measurements given known feature locations.

(c) Implement an EKF to map the features given known robot state trajectory. Use a *noiseless* ground-truth trajectory created by the given inputs as your reference. Start your initial mean positions at the origin, with a large covariance in the position ($\Sigma_0 = 10^5 I$)

(d) Combine the two items above to jointly estimate the robot pose and feature locations. This is EKF SLAM, the original SLAM solution.

For each of parts (b), (c), and (d), provide a plot with

- the mean estimate of the state (whether that is the robot's location, the feature positions, or both),
- the true trajectory and true positions of the features,
- the covariance ellipses around the mean position estimates.
- On a separate plot, include the Θ time series estimate, the true Θ trajectory, and the 95% confidence bounds around the estimate.

1. [15 points] **State and Parameter Estimation:** The EKF can be used to estimate unknown parameters of a dynamical system, while also estimating its state. Consider the roll mode of a fixed-wing aircraft, with the model $\dot{p} = -ap + bu$, where p is roll rate, u is an aileron input, and a, b are unknown (fixed) model parameters. You receive a measurement $y(t) = p(t)$ from the IMU.

   (a) Formulate a discrete time approximation of this system $x_{t+1} = f(x_t, u_t) + w_t$, $y_t = g(x_t) + v_t$ with the state $x = [p \quad a \quad b]^T$.

   (b) Find the Jacobians A_t and C_t for the EKF.

   (c) Simulate the system with true parameters $a = 0.5$, $b = 2$, with an input $u_t = \sin(t)$, with $p(0) = 5$, and with additive process noise covariance $Q = \text{diag}([0.1 \quad 0 \quad 0])$ and measurement noise covariance $R = 0.1$. Implement an EKF to estimate the parameters a and b, as well as p. Use the noise covariances $Q = \text{diag}([0.1 \quad 0.01 \quad 0.01])$, $R = 0.1$ in your EKF. Start with $\Sigma_0 = 10I_3$ and $\mu_0 = 0$. Use $dt = 0.1s$, and simulate for 100 seconds.
   Remark: Think about what this result represents. You only have a model of your system, not an exact specification. But this method lets you estimate from measurements, in real-time, not just your real state but also the parameters of the system!

a)

$f(x_t, u_t)$:
$p, a, b = x_t$
$\dot{p} = -ap + bu_t$
$\dot{x} = [\dot{p} \; 0 \; 0]$
return $x_t + \dot{x} \cdot dt$

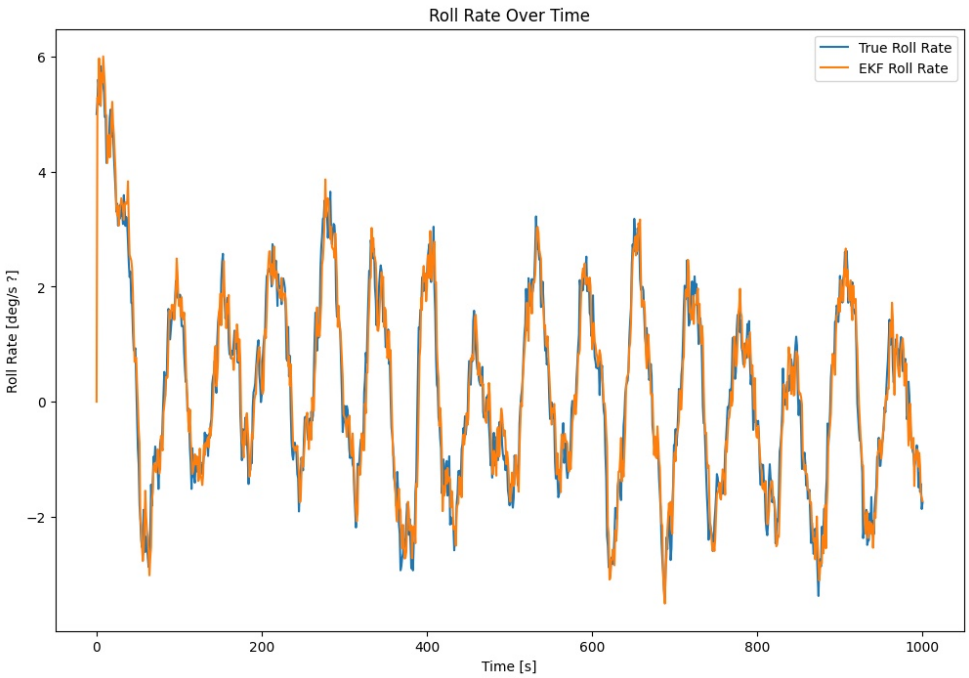
$g(x_t)$:
$p, a, b = x_t$
return p

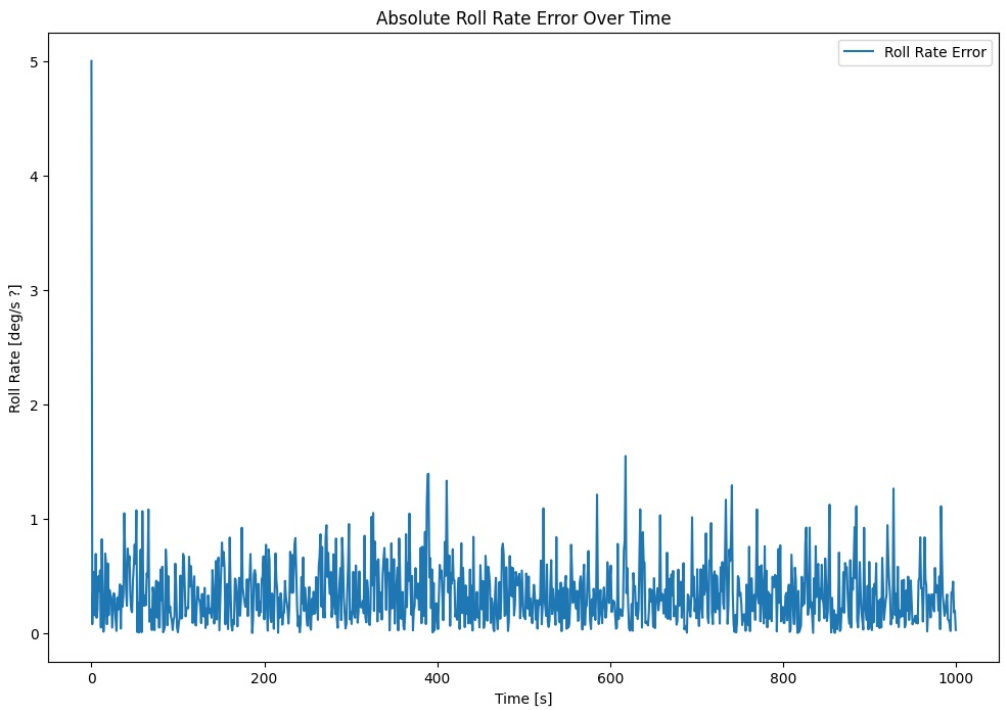
b)

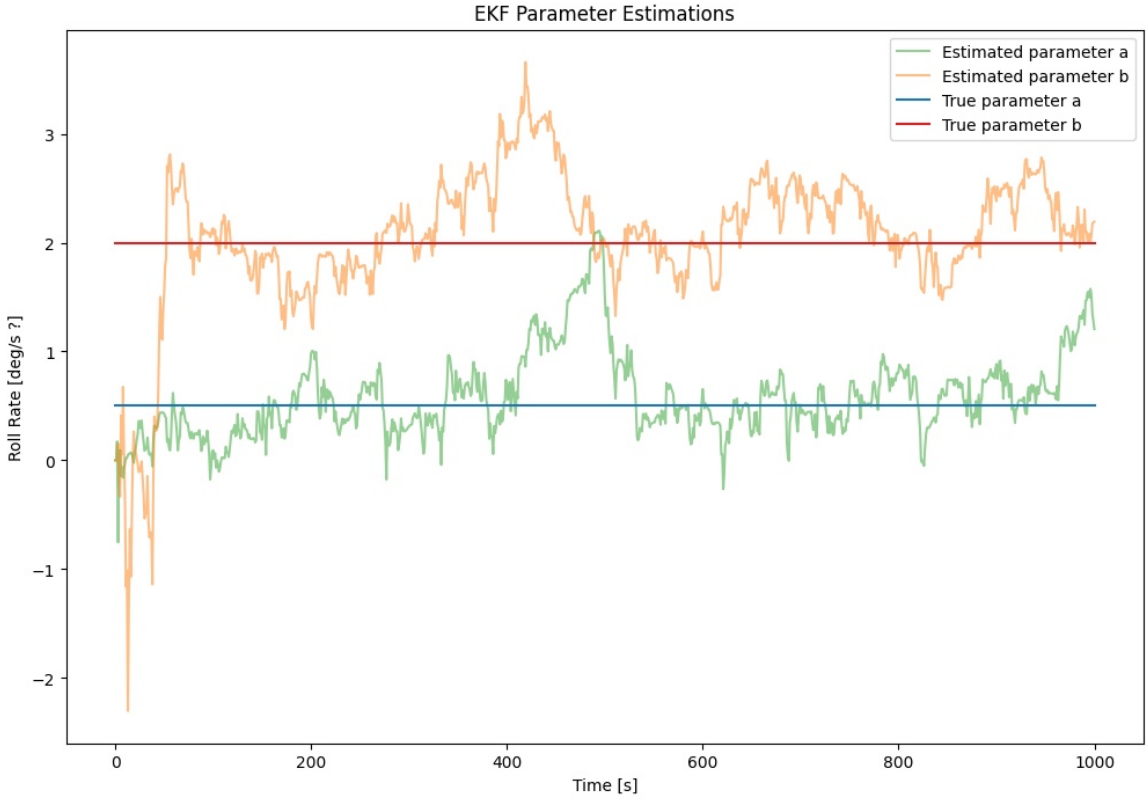
$$A = dt \begin{bmatrix} -a & -p & u \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

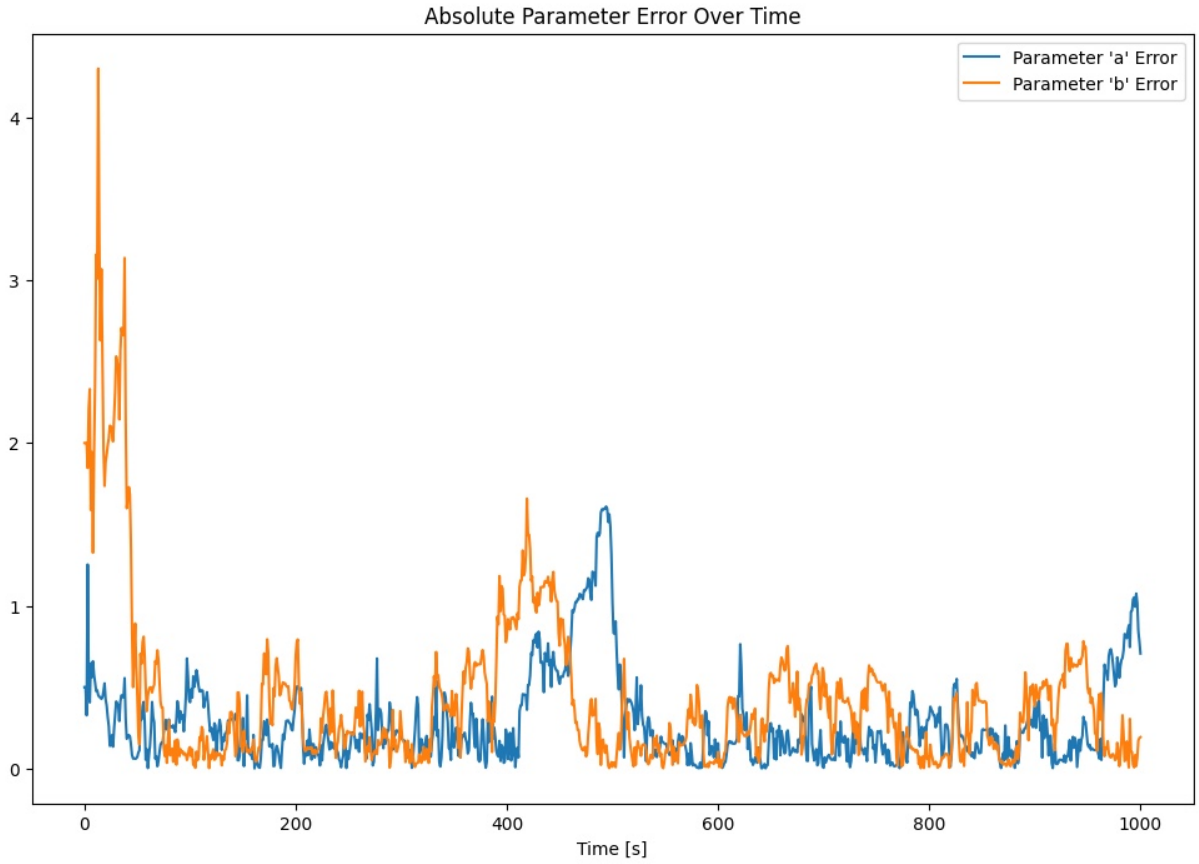
$$C = [1 \; 0 \; 0]$$

c)






EKF Parameter Estimations
Estimated parameter a
Estimated parameter b
True parameter a
True parameter b
Roll Rate [deg/s ?]
Time [s]
3
2
1
0
−1
−2
0
200
400
600
800
1000


Absolute Parameter Error Over Time
Parameter 'a' Error
Parameter 'b' Error
Time [s]
4
3
2
1
0
0
200
400
600
800
1000

2. [15 points] **Kalman Filter as an Optimization Problem**: In class we claimed that the Kalman Filter update equations can be derived as the analytical solution to a least squares optimization problem. Let's prove the claim. Consider the Maximum A Posteriori (MAP) optimization problem we saw in class, $x_t^{\text{MAP}} = \arg\min_{x_t} J(x_t)$. With a linear measurement model $Y_t = C_t X_t + V_t$, $V_t \sim \mathcal{N}(0, R_t)$, and Gaussian prediction distribution $p(x_t \mid y_{1:t-1}) = \mathcal{N}(\mu_{t|t-1}, \Sigma_{t|t-1})$, we saw that the MAP objective function becomes a convex quadratic in x_t,

$$J(x_t) = \frac{1}{2}\|y_t - C_t x_t\|^2_{R_t^{-1}} + \frac{1}{2}\|x_t - \mu_{t|t-1}\|^2_{\Sigma^{-1}_{t|t-1}}.$$

(a) Analytically find x_t^{MAP} by setting the gradient of the objective function to zero $\frac{\partial J(x_t)}{\partial x_t}|_{x_t = x_t^{\text{MAP}}} = 0$ and solving for x_t^{MAP}. Verify that $\mu_{t|t} = x_t^{\text{MAP}}$ by showing that your expression matches the mean update equation from the Kalman filter.

(b) Analytically find the Hessian, $H = \frac{\partial^2 J(x_t)}{\partial x_t^2}|_{x_t = x_t^{\text{MAP}}}$. Show that $\Sigma_{t|t} = H^{-1}$ by verifying that the inverse of your expression for H matches the covariance update equation from the Kalman Filter.

Hint: the matrix inverse lemmas that we have seen in previous problem sets will come in handy for this problem!

quadratic form derivative!

$$\frac{d}{dx}(x^T A x) = x^T(A + A^T)$$

matrix derivative

$$\frac{\partial A(y)}{\partial x} = \frac{\partial y}{\partial x}\frac{\partial A}{\partial y}$$

$$\|y_t - C_t x_t\|^2_{R_t^{-1}} = (y_t - C_t x_t)^T R_t^{-1}(y_t - C_t x_t)$$

$$\frac{\partial f}{\partial x} = (\nabla_x f)^T$$

$$\nabla_x J(x_t) = \frac{1}{2}\nabla_x \|y_t - C_t x_t\|^2_{R_t^{-1}} + \frac{1}{2}\|x_t - \mu_{t|t-1}\|^2_{\Sigma^{-1}_{t|t-1}}$$

(1)

$$\frac{\partial}{\partial x}\|y_t - C_t x_t\|^2_{R_t^{-1}} = \left[(y_t - C_t x_t)^T (R_t^{-1} + R_t^{-1})\right]\frac{\partial}{\partial x}(y_t - C_t x_t)$$

(R_t^{-1} symmetric)

$$= 2(y_t - C_t x_t)^T R_t^{-1}(-C_t) \rightarrow \nabla_x f_1 = \frac{\partial f_1}{\partial x}^T = -2C_t^T R_t^{-1}(y_t - C_t x_t)$$

(2)

$$\frac{\partial}{\partial x}\|x_t - \mu_{t|t-1}\|^2_{\Sigma^{-1}_{t|t-1}} = \left[(x_t - \mu_{t|t-1})^T(\Sigma^{-1}_{t|t-1} + \Sigma^{-1}_{t|t-1})\right]\frac{\partial}{\partial x}(x_t - \mu_{t|t-1})$$

$$= 2(x_t - \mu_{t|t-1})^T \Sigma^{-1}_{t|t-1} \qquad \nabla_x f_2 = \frac{\partial f_2}{\partial x}^T = 2\Sigma^{-1}_{t|t-1}(x - \mu_{t|t-1})$$

$$\therefore \quad \nabla_x J(x_t) = -C_t^T R_t^{-1}(y_t - C_t x_t) + \Sigma^{-1}_{t|t-1}(x - \mu_{t|t-1}) = 0$$

$$-C_t^T R_t^{-1}(y_t - C_t x_t) + \Sigma^{-1}_{t|t-1}(x - \mu_{t|t-1}) = 0$$

$$C_t^T R_t^{-1}(y_t - C_t x_t) = \Sigma^{-1}_{t|t-1}(x - \mu_{t|t-1})$$

$$C_t^T R_t^{-1} y_t - C_t^T R_t^{-1} C_t x_t = \Sigma^{-1}_{t|t-1} x_t - \Sigma^{-1}_{t|t-1}\mu_{t|t-1}$$

$$(C_t^T R_t^{-1} C_t + \Sigma^{-1}_{t|t-1}) x_t = (C_t^T R_t^{-1} y_t + \Sigma^{-1}_{t|t-1}\mu_{t|t-1})$$

$$x_t = (C_t^T R_t^{-1} C_t + \Sigma^{-1}_{t|t-1})^{-1}(C_t^T R_t^{-1} y_t + \Sigma^{-1}_{t|t-1}\mu_{t|t-1})$$

$$x_t = (C_t^T R_t^{-1} C_t + \Sigma^{-1}_{t|t-1})^{-1}(C_t^T R_t^{-1} y_t + \Sigma^{-1}_{t|t-1}\mu_{t|t-1})$$

$$C_t^T R_t^{-1} y_t + \Sigma^{-1}_{t|t-1}\mu_{t|t-1} + C_t^T R_t C_t \mu_{t|t-1} - C_t^T R_t C_t \mu_{t|t-1}$$

$$(C_t^T R_t^{-1} C_t + \Sigma_{t|t-1})\mu_{t|t-1} + C_t^T R_t^{-1}(y_t - C_t \mu_{t|t-1})$$

$$x_t = (C_t^T R_t^{-1} C_t + \Sigma^{-1}_{t|t-1})^{-1}\left[(C_t^T R_t^{-1} C_t + \Sigma_{t|t-1})\mu_{t|t-1} + C_t^T R_t^{-1}(y_t - C_t \mu_{t|t-1})\right]$$

$$x_t = \mu_{t|t-1} + \left(C_t^T R_t^{-1} C_t + \Sigma_{t|t-1}^{-1}\right)^{-1} C_t^T R_t^{-1}(y_t - C_t \mu_{t|t-1})$$

prev. HW

$$(A+UCV)^{-1}UC = A^{-1}U(C^{-1}+VA^{-1}U)^{-1}.$$

$$x_t = \mu_{t|t-1} + \Sigma_{t|t-1} C_t^T \left(R_t + C_t \Sigma_{t|t-1} C_t^T\right)^{-1}(y_t - C_t \mu_{t|t-1})$$

same as $\mu_{t|t}$ in KF update

$$\nabla_x J(x_t) = -C_t^T R_t^{-1}(y_t - C_t x_t) + \Sigma_{t|t-1}^{-1}(x - \mu_{t|t-1})$$

$$\nabla_x^2 J(x_t) = C_t^T R_t^{-1} C_t + \Sigma_{t|t-1}^{-1} = H$$

$$H^{-1} = \left(C_t^T R_t^{-1} C_t + \Sigma_{t|t-1}^{-1}\right)^{-1}$$

prev HW

$$(A+UCV)^{-1} = A^{-1} - A^{-1}U(VA^{-1}U + C^{-1})^{-1}VA^{-1}$$

$$H^{-1} = \Sigma_{t|t-1} - \Sigma_{t|t-1} C_t^T \left(C_t \Sigma_{t|t-1} C_t^T + R_t\right)^{-1} C_t \Sigma_{t|t-1}$$

same as $\Sigma_{t|t}$ in KF update

3. [20 points] **EKF SLAM**: Consider again the differential drive robot model,

$$P_{t+1}^x = P_t^x + \delta t\, s_t \cos\Theta_t + W_t^x$$
$$P_{t+1}^y = P_t^y + \delta t\, s_t \sin\Theta_t + W_t^y$$
$$\Theta_{t+1} = \Theta_t + \delta t\, \phi_t + W_t^\theta$$

where $P_t = [P_t^x \quad P_t^y]^T$ is the robot position relative to a feature point, Θ_t is its heading angle, s_t its speed input, ϕ_t its rotation rate input, and W_t is the process noise distributed as $W_t \sim \mathcal{N}(0, Q)$ with $Q = 0.1\delta t I_3$. Suppose you have four features at positions $m_1 = [0 \quad 0]^T$, $m_2 = [10 \quad 0]^T$, $m_3 = [10 \quad 10]^T$, and $m_4 = [0 \quad 10]^T$. Now suppose the robot takes relative position measurements to each of the features (in the robot's local frame), with measurement model,

$$Y_t^j = \begin{bmatrix} \cos(\Theta_t) & \sin(\Theta_t) \\ -\sin(\Theta_t) & \cos(\Theta_t) \end{bmatrix} (m_j - P_t) + V_t^j,$$

as we saw in class. The measurements have an associated measurement uncertainty of $V_t^j \sim \mathcal{N}(0, R)$, where $R = 0.25 I_2$. Also suppose that the robot pose is initially distributed $\mathcal{N}([1m \; 1m \; 0rad]^T, I_3)$ is commanded with a known control sequence $s_t = 0.5$ and $\phi_t = \sin(2t)$rad/s. You can assume that the ground truth trajectory also starts at $[1m \; 1m \; 0rad]^T$. Use $\delta t = 0.5s$ and run for 100 timesteps (i.e., $50s$).

We highly recommend using the starter code from Problem Set 4 Q4, and editing it as necessary for this problem.

(a) Calculate the measurement Jacobian C_t^j for the measurement model.

(b) Implement an EKF for the robot to localize itself with these measurements given known feature locations.

(c) Implement an EKF to map the features given known robot state trajectory. Use a *noiseless* ground-truth trajectory created by the given inputs as your reference. Start your initial mean positions at the origin, with a large covariance in the position ($\Sigma_0 = 10^5 I$)

(d) Combine the two items above to jointly estimate the robot pose and feature locations. This is EKF SLAM, the original SLAM solution.

For each of parts (b), (c), and (d), provide a plot with

- the mean estimate of the state (whether that is the robot's location, the feature positions, or both),
- the true trajectory and true positions of the features,
- the covariance ellipses around the mean position estimates.
- On a separate plot, include the Θ time series estimate, the true Θ trajectory, and the 95% confidence bounds around the estimate.

a) $A = \begin{bmatrix} 1 & 0 & -\delta t \cdot s \cdot \sin(\theta) \\ 0 & 1 & \delta t \cdot s \cdot \cos(\theta) \\ 0 & 0 & 1 \end{bmatrix}$

$C = \begin{bmatrix} \frac{\partial y_x^1}{\partial x} & \frac{\partial y_x^1}{\partial y} & \frac{\partial y_x^1}{\partial \theta} \\ \vdots & \vdots & \vdots \end{bmatrix}$

$y^j = \begin{bmatrix} c\theta & s\theta \\ -s\theta & c\theta \end{bmatrix} (m_j - p_t)$

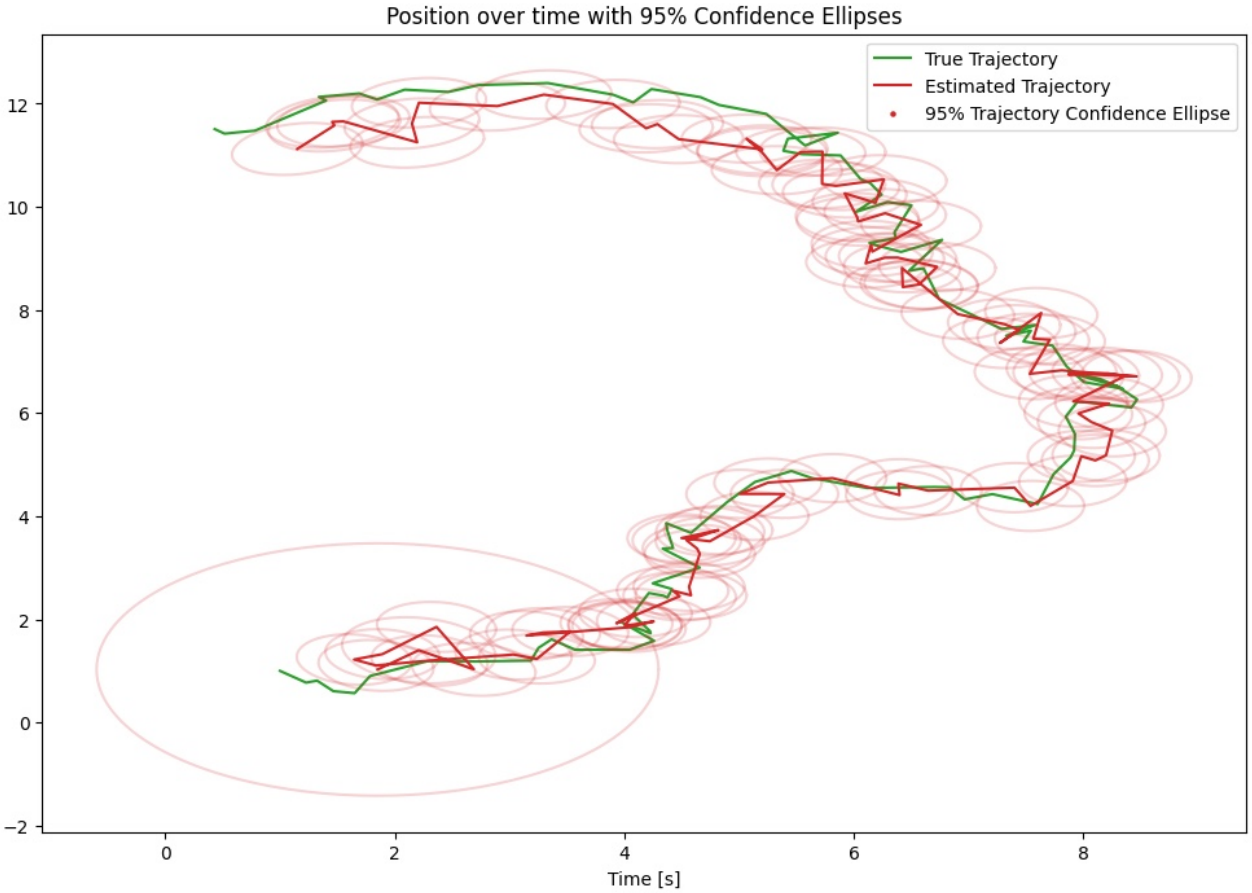
$\frac{\partial y^j}{\partial x} = \begin{bmatrix} c\theta & s\theta \\ -s\theta & c\theta \end{bmatrix} \frac{\partial}{\partial x}(m_j - p_t) = -\begin{bmatrix} c\theta & s\theta \\ -s\theta & c\theta \end{bmatrix} \frac{\partial p_t}{\partial x} = -\begin{bmatrix} c\theta & s\theta \\ -s\theta & c\theta \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} -c\theta \\ s\theta \end{bmatrix}$

$\frac{\partial y^j}{\partial y} = \begin{bmatrix} c\theta & s\theta \\ -s\theta & c\theta \end{bmatrix} \frac{\partial}{\partial y}(m_j - p_t) = -\begin{bmatrix} c\theta & s\theta \\ -s\theta & c\theta \end{bmatrix} \begin{bmatrix} 0 \\ -1 \end{bmatrix} = \begin{bmatrix} -s\theta \\ -c\theta \end{bmatrix}$

$\frac{\partial y^j}{\partial \theta} = \begin{bmatrix} -s\theta & c\theta \\ -c\theta & -s\theta \end{bmatrix} (m_j - p_t) = \begin{bmatrix} -s\theta (m_{j,x} - x) + c\theta (m_{j,y} - y) \\ -c\theta (m_{j,x} - x) - s\theta (m_{j,y} - y) \end{bmatrix}$

$C_t = \begin{bmatrix} -c\theta & -s\theta & -s\theta (m_{1,x} - x) + c\theta (m_{1,y} - y) \\ s\theta & c\theta & -c\theta (m_{1,x} - x) - s\theta (m_{1,y} - y) \\ -c\theta & -s\theta & -s\theta (m_{2,x} - x) + c\theta (m_{2,y} - y) \\ s\theta & c\theta & -c\theta (m_{2,x} - x) - s\theta (m_{2,y} - y) \\ -c\theta & -s\theta & -s\theta (m_{3,x} - x) + c\theta (m_{3,y} - y) \\ s\theta & c\theta & -c\theta (m_{3,x} - x) - s\theta (m_{3,y} - y) \\ -c\theta & -s\theta & -s\theta (m_{4,x} - x) + c\theta (m_{4,y} - y) \\ s\theta & c\theta & -c\theta (m_{4,x} - x) - s\theta (m_{4,y} - y) \end{bmatrix}$

5)



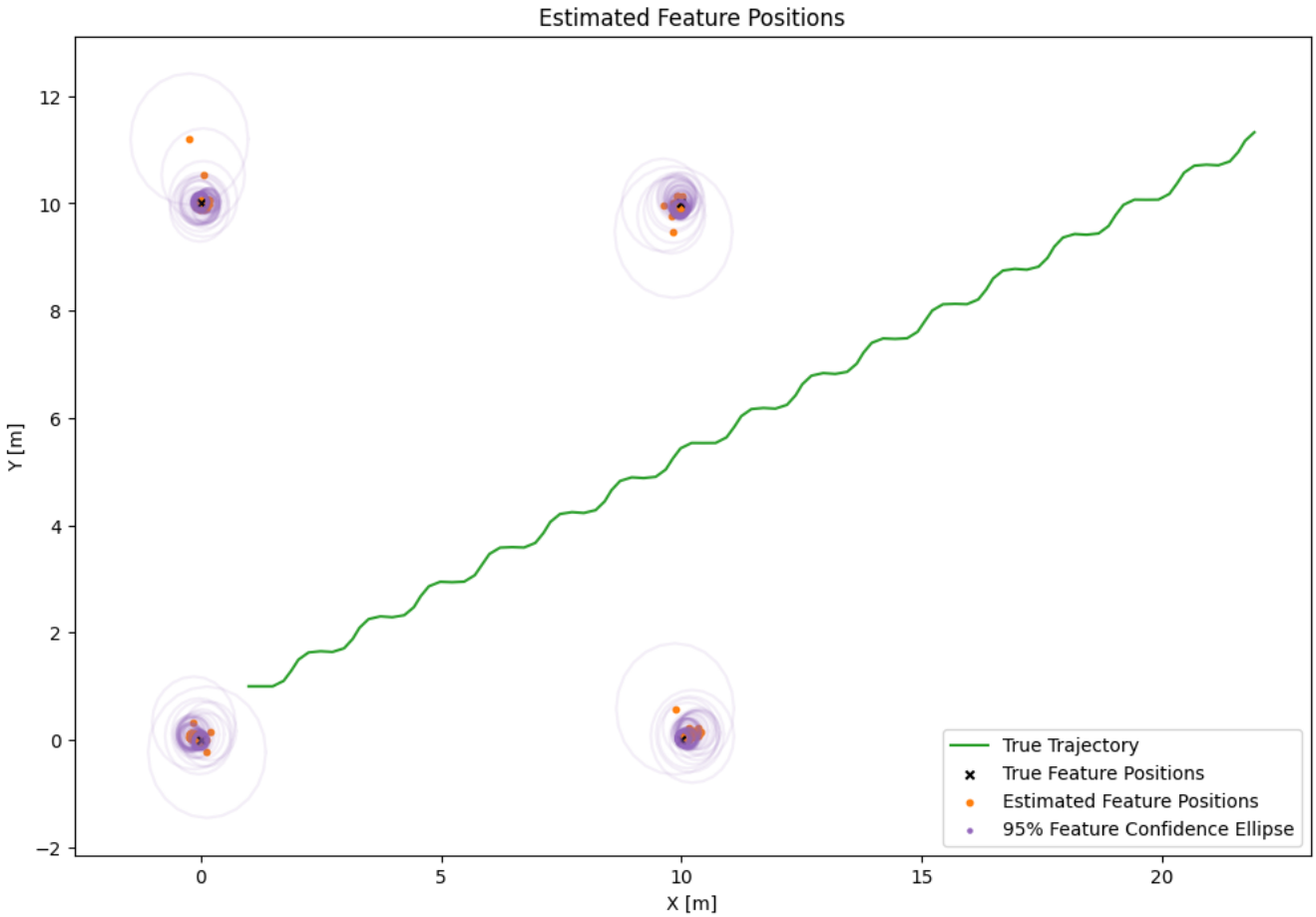
c)

$f[x,u] = 0 \quad A = I_6$ since features do not change position

$$y^j = \begin{bmatrix} c\theta & s\theta \\ -s\theta & c\theta \end{bmatrix} (m_j - p_t) \qquad m_j - p_t = \begin{bmatrix} c\theta & -s\theta \\ s\theta & c\theta \end{bmatrix} y^j$$

$$m_j = p_t + \begin{bmatrix} c\theta & -s\theta \\ s\theta & c\theta \end{bmatrix} y^j$$

$$\frac{\partial y^j}{\partial m_j} = \begin{bmatrix} c\theta & s\theta \\ -s\theta & c\theta \end{bmatrix} \qquad C = \begin{bmatrix} c\theta & s\theta & \cdots & \cdots & \cdots \\ -s\theta & c\theta & & & \\ \vdots & & c\theta & s\theta & \vdots \\ & & -s\theta & c\theta & \\ \cdots & \cdots & \cdots & c\theta & s\theta \\ \vdots & \vdots & \vdots & -s\theta & c\theta \end{bmatrix}$$



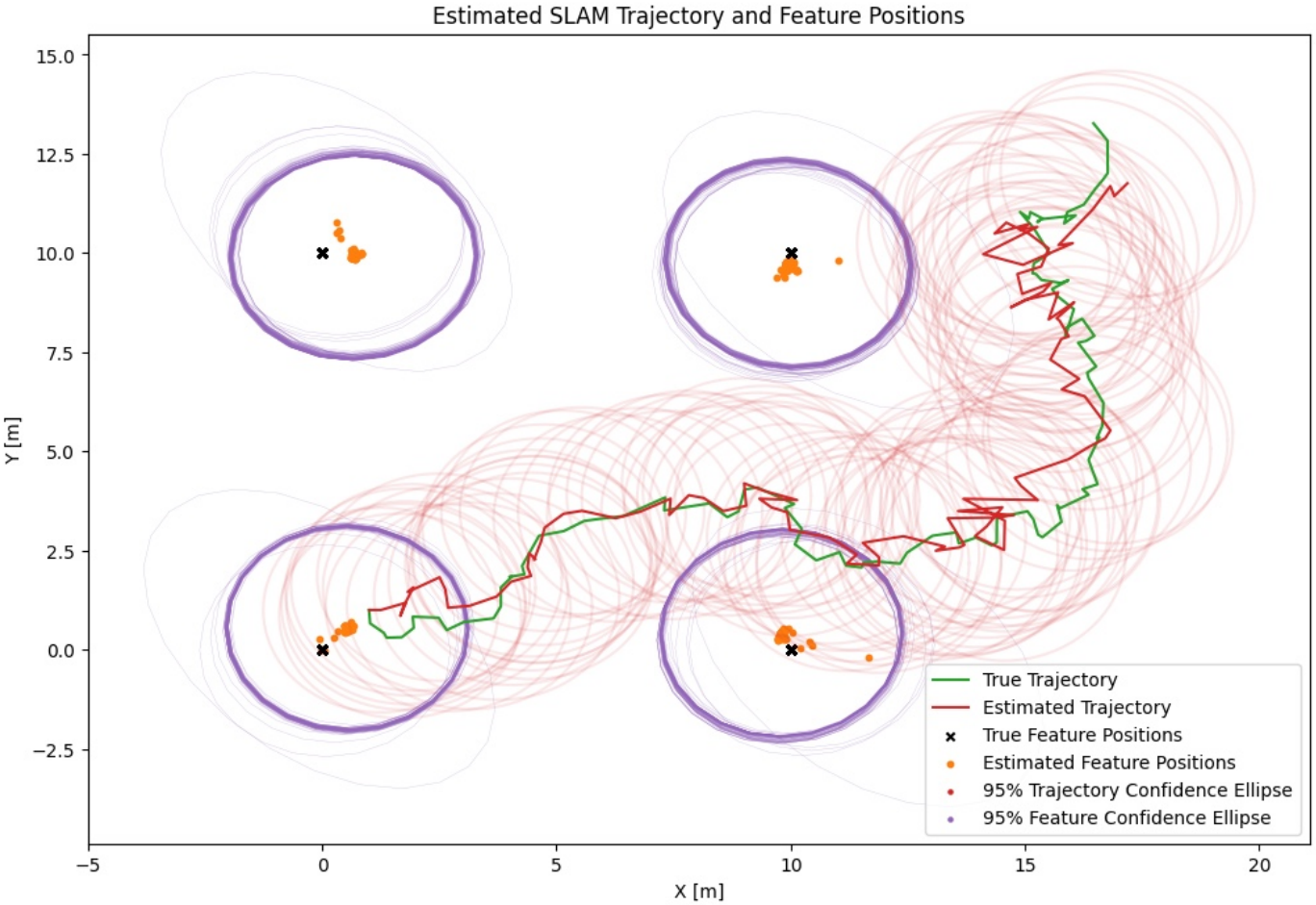
d) $X = (p_x, p_y, p_\theta, m_{x1}, m_{y1}, m_{x2}, m_{y2}, m_{x3}, m_{y3}, m_{x4}]^T = [x, m]^T$

$f(x, u, \delta t)$

$s, \phi = u$

$$x_{t+1} = \begin{bmatrix} p_{t+1} \\ m_{t+1} \end{bmatrix} = \begin{bmatrix} p_x + \delta t \cdot s_t \cdot \cos\theta \\ p_y + \delta t \cdot s_t \cdot \sin\theta \\ p_\theta + \delta t \cdot \phi \\ m_{ix} \\ m_{iy} \\ \vdots \end{bmatrix}$$

$$A = \begin{bmatrix} A_{robot} & 0 \\ 0 & A_{features} \end{bmatrix} \qquad C = \begin{bmatrix} C_{robot} & C_{features} \end{bmatrix}$$


Estimated SLAM Trajectory and Feature Positions
15.0
12.5
10.0
7.5
5.0
2.5
0.0
−2.5
Y [m]
−5
0
5
10
15
20
X [m]
True Trajectory
Estimated Trajectory
True Feature Positions
Estimated Feature Positions
95% Trajectory Confidence Ellipse
95% Feature Confidence Ellipse

wow!

In [540…
```
import numpy as np
from numpy import sin, cos, sqrt
import matplotlib.pyplot as plt
from numpy.linalg import inv, cholesky, norm
from scipy.linalg import sqrtm
from time import time
```

In [541…
```
def noise(cov):
    return np.random.multivariate_normal(
        mean=np.zeros_like(cov[0]),
        cov=cov
    )
```

In [542…
```
def get_ellipse_points(mu: np.ndarray, Sigma: np.ndarray, P=0.95, N=200):
    t = np.linspace(0, 2*np.pi, N)
    circle = np.vstack((np.cos(t), np.sin(t)))
    rhs = -2*np.log(1-P)
    ellipse = np.sqrt(rhs) * circle
    scaled_ellipse = sqrtm(Sigma) @ ellipse # circle -> ellipse
    return scaled_ellipse + mu.reshape(2,1)
```

Problem 1

EKF

In [543…
```
def f_1(X, u, dt):
    p,a,b = X
    p_dot = -a*p + b*u
    X_dot = np.array([p_dot, 0, 0])
    return X + X_dot*dt

def g_1(X):
    return X[0]
```

In [544…
```
def get_A_1(X, u, dt):
    p,a,b, = X
    # return dt * np.array([-a, -p, u])
    return np.array([
        [1-dt*a, -dt*p, dt*u],
        [0, 1, 0],
        [0, 0, 1]
    ])

def get_C_1():
    return np.array([1, 0, 0]).reshape((1,3))
```

In [545…
```
a = 0.5
b = 2
```

```
X0 = np.array([5, a, b])

def get_u(t):
    return sin(t)

Q_true = np.diag([0.1, 0, 0])
R_true = np.eye(1)*0.1

dt = 0.1
t_max = 100
N = int(t_max/dt)
t_arr = np.linspace(0, t_max, N+1)
```

In [546…
```
mu0 = np.array([0, 0, 0])
Sigma0 = np.eye(3)*10

Q_ekf = np.diag([0.1, 0.01, 0.01])
R_ekf = np.eye(1)*0.1
```

In [547…
```
# np.array([noise(R_true), 0, 0])
# np.array([noise(R_true).reshape(1)[0], 0, 0])
noise(Q_true)
```

Out[547… array([-0.0672323, 0.   , 0.   ])

In [548…
```
X_arr = np.zeros((N+1, 3))
Y_arr = np.zeros((N+1, 1))

mu_arr = np.zeros((N+1, 3))
Sigma_arr = np.zeros((N+1, 3, 3))

X_arr[0] = X0
Y_arr[0] = X0[0] + noise(R_true)

mu_arr[0] = mu0
Sigma_arr[0] = Sigma0

t_arr = dt*np.arange(N+1)

for i in range(N):
    t = t_arr[i]
    # True
    X = X_arr[i]
    u = get_u(t)

    X_arr[i+1] = f_1(X_arr[i], u, dt) + noise(Q_true)
    Y_arr[i] = g_1(X_arr[i]) + noise(R_true)

    # Measure


    # EKF Predict
    mu_pred = f_1(mu_arr[i], u, dt)
    A = get_A_1(mu_arr[i], u, dt)
    Sigma_predict = A @ Sigma_arr[i] @ A.T + Q_ekf
```

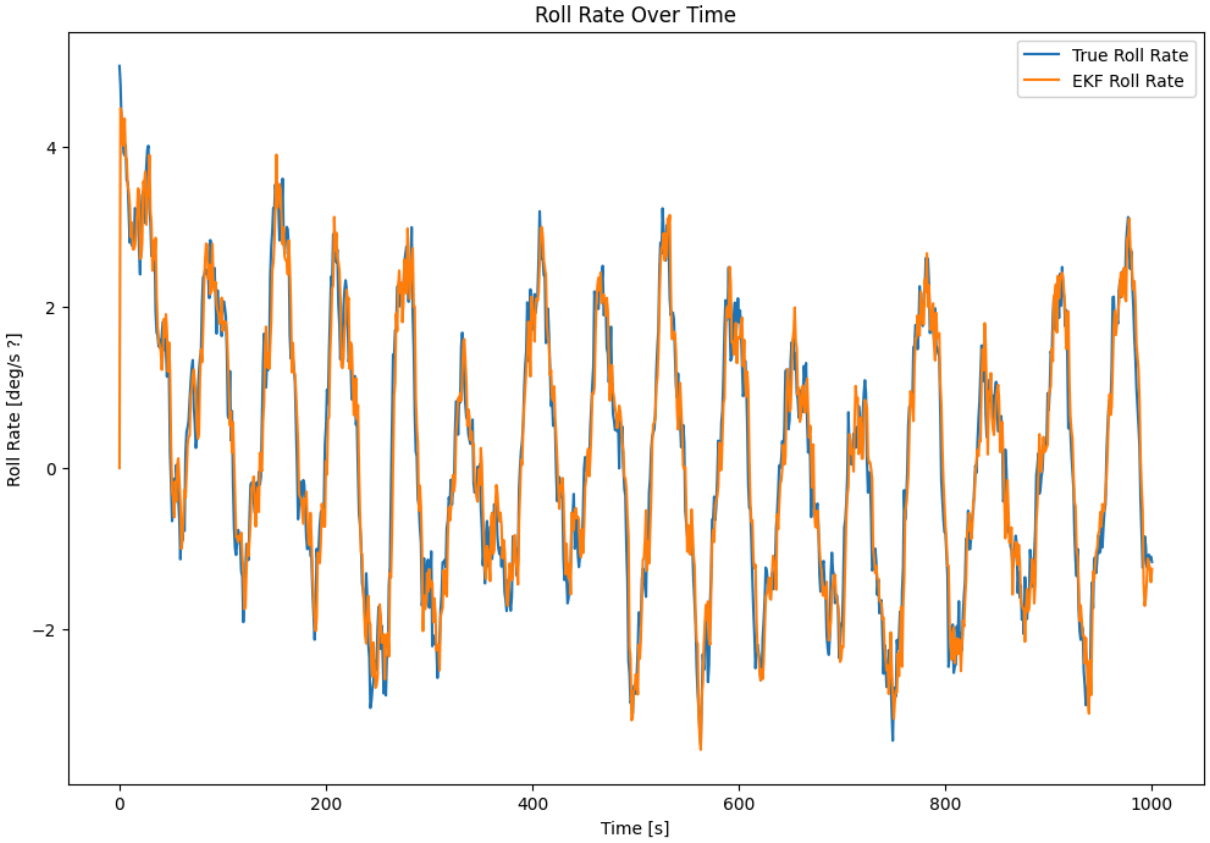
```
    # EKF Update
    C = get_C_1()
    K = Sigma_predict @ C.T @ inv(C @ Sigma_predict @ C.T + R_ekf)
    mu_arr[i+1] = mu_pred + K @ (Y_arr[i] - g_1(mu_pred))

    Sigma_arr[i+1] = Sigma_predict - K @ C @ Sigma_predict
```

In [549…

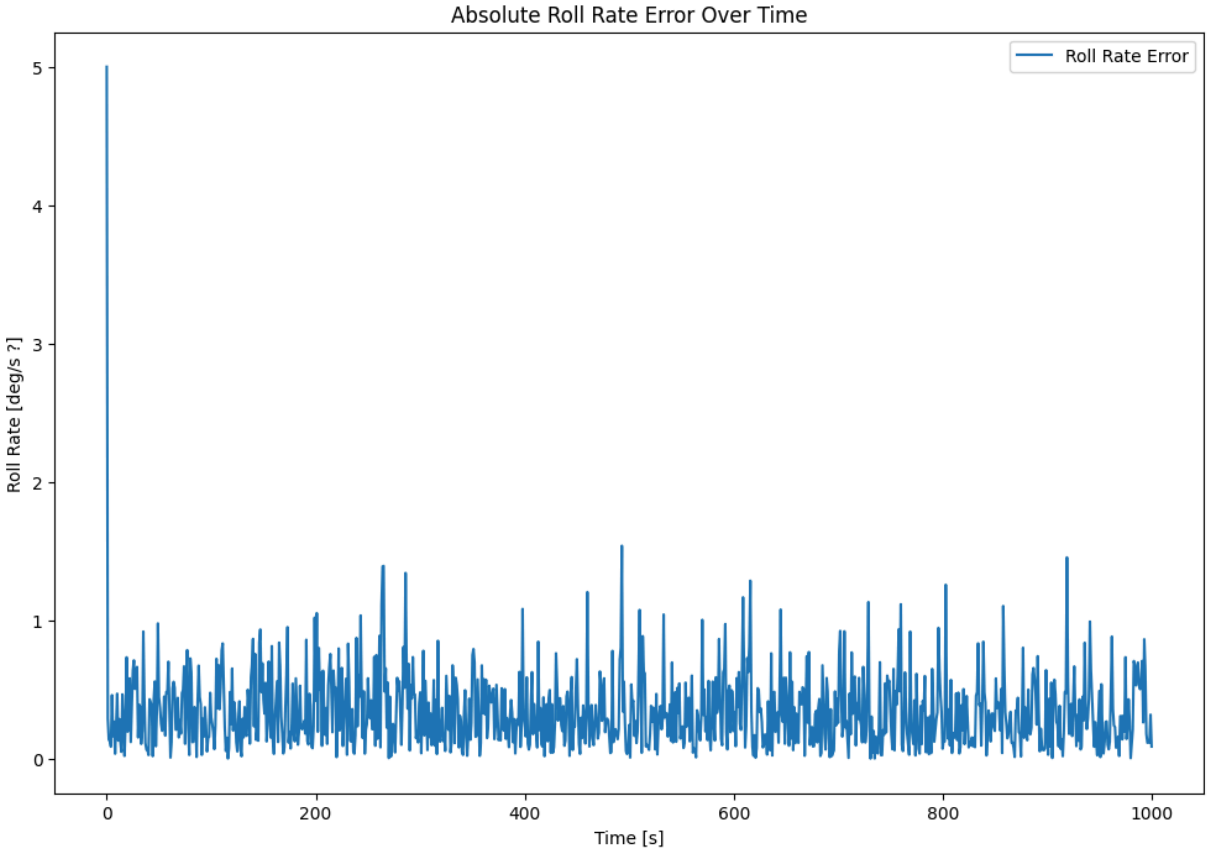
```
plt.figure(figsize=(12,8))
plt.plot(X_arr[:,0], label="True Roll Rate")
plt.plot(mu_arr[:,0], label="EKF Roll Rate")

plt.title("Roll Rate Over Time")
plt.xlabel("Time [s]")
plt.ylabel("Roll Rate [deg/s ?]")
plt.legend()
plt.show()
```



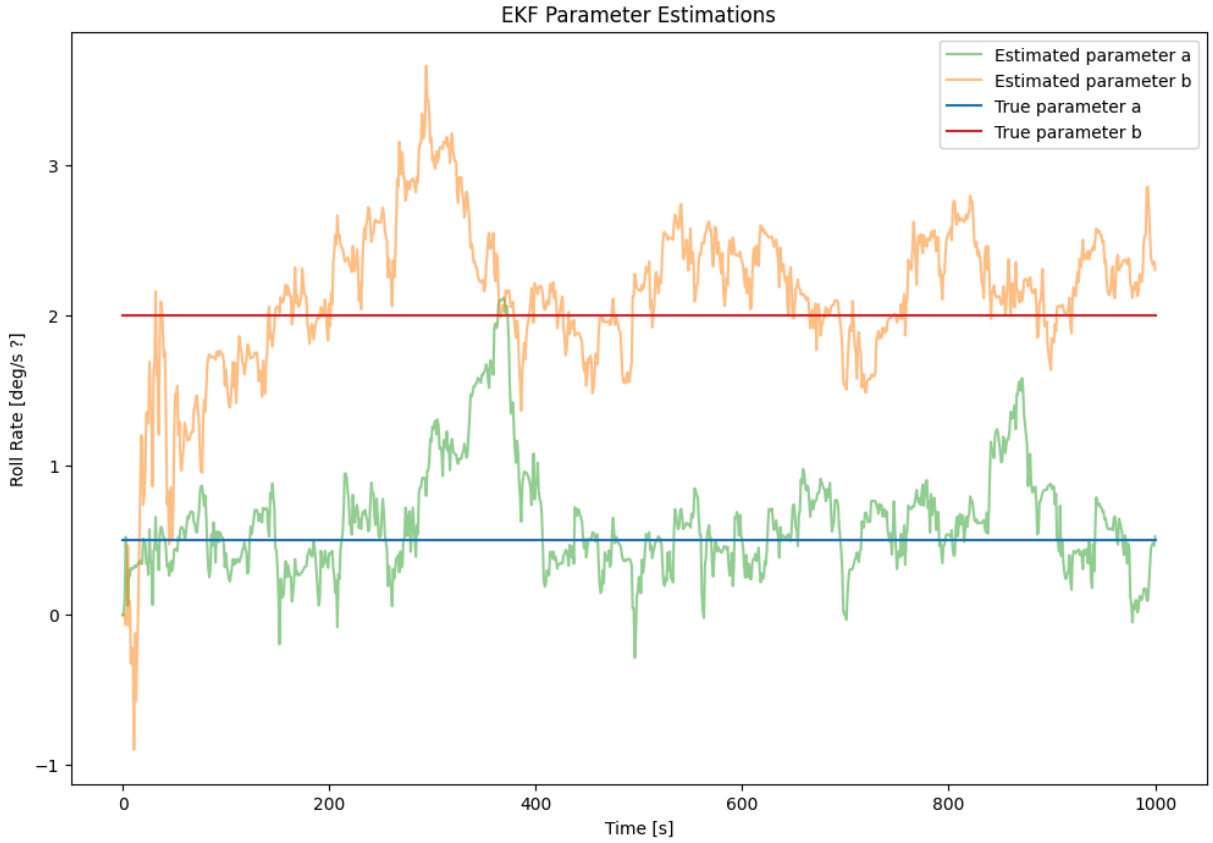
In [550…

```
plt.figure(figsize=(12,8))
plt.plot(abs(X_arr[:,0] - mu_arr[:,0]), label="Roll Rate Error")
plt.title("Absolute Roll Rate Error Over Time")
plt.xlabel("Time [s]")
plt.ylabel("Roll Rate [deg/s ?]")
plt.legend()
plt.show()
```



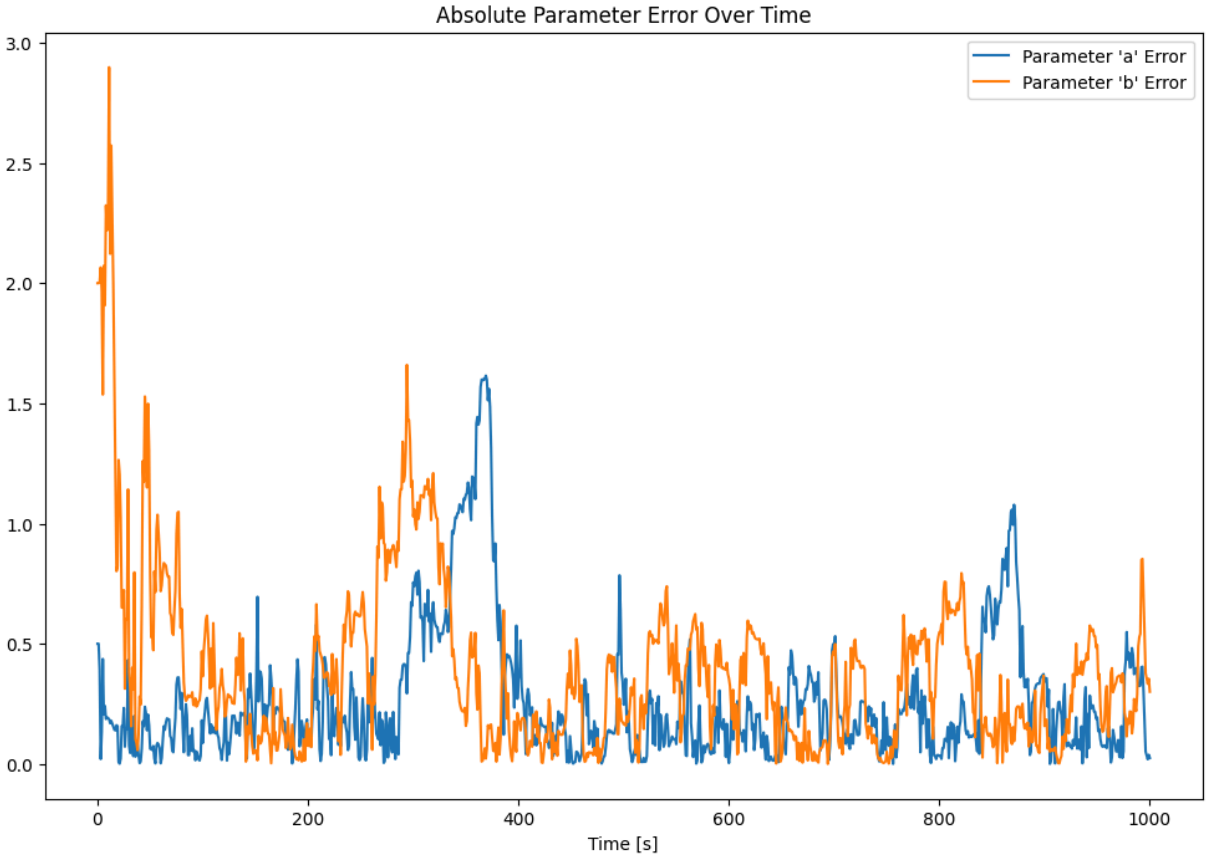
In [551…

```
plt.figure(figsize=(12,8))
plt.plot(mu_arr[:,1],label="Estimated parameter a", alpha=0.5, color="tab:gr
plt.plot(mu_arr[:,2],label="Estimated parameter b", alpha=0.5, color="tab:or
plt.plot(X_arr[:,1],label="True parameter a", color="tab:blue")
plt.plot(X_arr[:,2],label="True parameter b", color="tab:red")
plt.title("EKF Parameter Estimations")
plt.xlabel("Time [s]")
plt.ylabel("Roll Rate [deg/s ?]")
plt.legend()
plt.show()
```



In [552…

```
plt.figure(figsize=(12,8))
plt.plot(abs(mu_arr[:,1] - X_arr[:,1]), label="Parameter 'a' Error")
plt.plot(abs(mu_arr[:,2] - X_arr[:,2]), label="Parameter 'b' Error")
plt.title("Absolute Parameter Error Over Time")
plt.xlabel("Time [s]")
# plt.ylabel("Roll Rate [deg/s ?]")
plt.legend()
plt.show()
```



Problem 3

In [553…

```
def f_3(X, u, dt):
    x, y, Theta = X
    s, phi = u

    x_next = x + dt*s*cos(Theta)
    y_next = y + dt*s*sin(Theta)
    Theta_next = Theta + dt*phi

    return np.array([
        x_next, y_next, Theta_next
    ])

def g_3(X, positions):
    P = X[0:2]
    Theta = X[2]

    A = np.array([
        [cos(Theta), sin(Theta)],
        [-sin(Theta), cos(Theta)]])

    measurements = [(A @ (position - P)).T
        for position in positions]

    return np.asarray(measurements).reshape((8))
```

```
X = np.array([3,3,3])
measurement_locations = np.array([
    [0, 0], [10, 0], [10, 10], [0, 10]
])
```

In [554…
```
def get_A_3(X, u, dt):
    _, _, Theta = X[0:3]
    s, _ = u

    A = np.array([
        [1, 0, -dt*s*sin(Theta)],
        [0, 1,  dt*s*cos(Theta)],
        [0, 0,        1        ]
    ])
    return A

def get_C_3(X, positions):
    x, y, Theta = X

    c = cos(Theta)
    s = sin(Theta)

    rows = []
    for pos in positions:
        C = np.array([
            [-c, -s, -s*(pos[0] - x) + c*(pos[1] - y)],
            [s, -c, -c*(pos[0] - x) - s*(pos[1] - y)]
        ])

        rows.append(C)

    return np.vstack(rows)


X = np.array([10,30,40])
measurement_locations = np.array([
    [0, 0], [10, 0], [10, 10], [0, 10]
])

# get_C(X, measurement_locations)
```

In [555…
```
def get_u(t):
    return np.array([0.5, sin(2*t)])
```

In [556…
```
SEED = 273
```

In [557…
```
measurement_locations = np.array([
    [0, 0], [10, 0], [10, 10], [0, 10]
])

dt = 0.5
TIMESTEPS = 100
Q_robot = np.eye(3) * 0.1 * dt
R_M = np.eye(8) * 0.25 # for 8 measurements
```

```
np.random.seed(SEED)
X0_robot = np.array([1,1,0])
```

In [558…
```
mu0_pos = np.random.multivariate_normal(
        mean=X0_robot,
        cov=np.eye(3)
    )
# Sigma0 = np.eye(3)* 1e5
Sigma0_pos = np.eye(3)
```

Part b

In [559…
```
# np.random.seed(SEED)
np.random.seed(273)

X_arr = np.zeros((TIMESTEPS+1, 3))
Y_arr = np.zeros((TIMESTEPS+1, 8))

mu_arr = np.zeros((TIMESTEPS+1, 3))
Sigma_arr = np.zeros((TIMESTEPS+1, 3, 3))

X_arr[0] = X0_robot
Y_arr[0] = g_3(X0_robot, measurement_locations) + noise(R_M)

mu_arr[0] = mu0_pos
Sigma_arr[0] = Sigma0_pos

for i in range(TIMESTEPS):
    t = i * dt

    # True
    X = X_arr[i]
    u = get_u(t)

    X_arr[i+1] = f_3(X, u, dt) + noise(Q_robot)
    # print(g_prob3(X, measurement_locations))
    Y_arr[i] = g_3(X, measurement_locations) + noise(R_M)
    # break

    # Measure


    # EKF Predict
    mu_pred = f_3(mu_arr[i], u, dt)

    A = get_A_3(mu_arr[i], u, dt)
    Sigma_predict = A @ Sigma_arr[i] @ A.T + Q_robot



    # EKF Update
    C = get_C_3(mu_pred, measurement_locations)
```

```
    # print(C.shape)
    # print(Sigma_predict.shape)
    K = Sigma_predict @ C.T @ inv(C @ Sigma_predict @ C.T + R_M)
    mu_arr[i+1] = mu_pred + K @ (Y_arr[i] - g_3(mu_pred, measurement_locatic

    Sigma_arr[i+1] = Sigma_predict - K @ C @ Sigma_predict

# plt.figure(figsize=(12,8))
# plt.plot(X_arr[:,0], X_arr[:,1], label="Actual trajectory")
# plt.plot(mu_arr[:,0], mu_arr[:,1], label="Measured trajectory")
# # plt.plot(X_arr[:,0], label="EKF Y Pos")

# plt.title("Position over time")
# plt.xlabel("Time [s]")
# plt.legend()
# plt.show()
```

In [560…

```
plt.figure(figsize=(12,8))

plt.plot([], [], color="tab:green", label="True Trajectory")[0]
plt.plot([], [], label="Estimated Trajectory", color="tab:red")[0]
plt.scatter([], [], color='tab:red', zorder=2, s=5, label="95% Trajectory Co

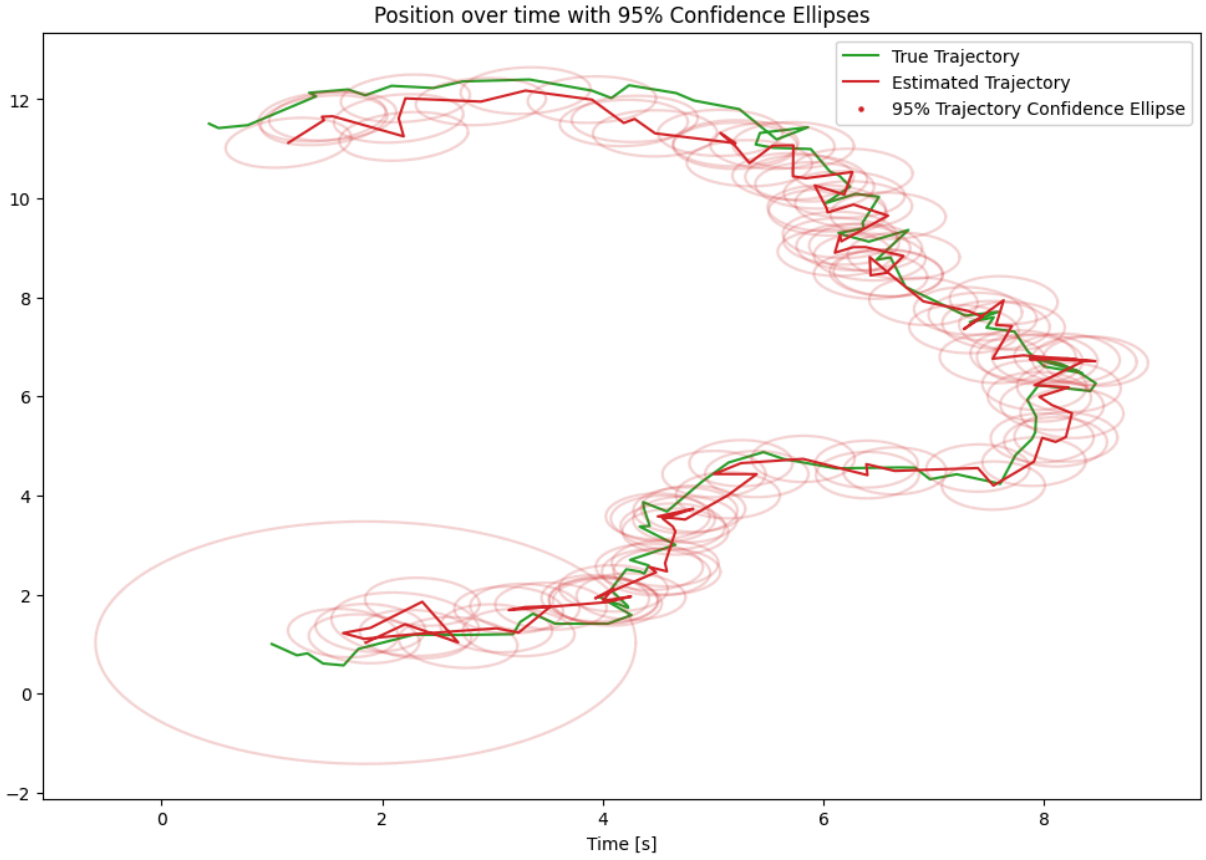


plt.plot(X_arr[:,0], X_arr[:,1], color="tab:green")
plt.plot(mu_arr[:,0], mu_arr[:,1], color="tab:red")

for mu, sigma in zip(mu_arr, Sigma_arr):
    position = mu[0:2]
    position_sigma = sigma[0:2, 0:2]
    factor = 1

    ellipse_x, ellipse_y = get_ellipse_points(position, position_sigma)
    plt.plot(ellipse_x, ellipse_y, color='tab:red', zorder=2, alpha=0.2)


plt.title("Position over time with 95% Confidence Ellipses")
plt.xlabel("Time [s]")
plt.legend()
plt.show()
```



part c

In [561…

```
def g_3_map(positions, robot_state):
    P = robot_state[0:2]
    Theta = robot_state[2]

    A_T = np.array([
        [cos(Theta), sin(Theta)],
        [-sin(Theta), cos(Theta)]])

    measurements = []

    for i in range(0, len(positions), 2):

        measurement = positions[i:i+2]
        measurements.append(A_T @ (measurement - P))

    return np.hstack(measurements)


def get_C_3_map(robot_state, n_landmarks):
    _, _, Theta = robot_state

    A = np.array([
        [cos(Theta), sin(Theta)],
        [-sin(Theta), cos(Theta)]])
```

```
    C = np.zeros((2*n_landmarks,2*n_landmarks))

    for i in range(0, 2*n_landmarks, 2):

        C[i:i+2, i:i+2] = A

    return C
```

In [562…
```
est_positions0 = np.zeros(8)
mu0_features = np.zeros(8)
Sigma0_features = np.eye(8) * 1e5
Q_F = np.zeros((8,8))
```

In [563…
```
X_arr = np.zeros((TIMESTEPS+1, 3))
Y_arr = np.zeros((TIMESTEPS+1, 8))

mu_arr = np.zeros((TIMESTEPS+1, 8))
Sigma_arr = np.zeros((TIMESTEPS+1, 8, 8))

X_arr[0] = X0_robot
Y_arr[0] = g_3_map(measurement_locations.reshape((8)), X0_robot) + noise(R_M

mu_arr[0] = mu0_features
Sigma_arr[0] = Sigma0_features

n_landmarks = len(measurement_locations)

for i in range(TIMESTEPS):
    t = i * dt

    # Trajectory
    u = get_u(t)

    X_arr[i+1] = f_3(X_arr[i], u, dt)
    # print(g_prob3(X, measurement_locations))
    Y_arr[i] = g_3_map(measurement_locations.reshape((8)), X_arr[i]) + noise

    # EKF Predict
    mu_pred = mu_arr[i]
    Sigma_predict = Sigma_arr[i] + Q_F



    # EKF Update
    C = get_C_3_map(X_arr[i], n_landmarks)

    K = Sigma_predict @ C.T @ inv(C @ Sigma_predict @ C.T + R_M)
    mu_arr[i+1] = mu_pred + K @ (Y_arr[i] - g_3_map(mu_pred, X_arr[i]))

    Sigma_arr[i+1] = Sigma_predict - K @ C @ Sigma_predict
```

In [564…
```
plt.figure(figsize=(12,8))

plt.plot([], [], color="tab:green", label="True Trajectory")[0]
plt.scatter([], [], label="True Feature Positions", s=20, color="k", marker=
```

```
plt.scatter([], [], label="Estimated Feature Positions", s=10, color="tab:or
plt.scatter([], [], color='tab:purple', s=5, label=r"95% Feature Confidence

for mu, sigma in zip(mu_arr[1:], Sigma_arr[1:]):

    for i in range(0, 8, 2):
        position = mu[i:i+2]
        position_sigma = sigma[i:i+2, i:i+2]
        factor = 1
        ellipse_x, ellipse_y = get_ellipse_points(position, position_sigma,
        plt.plot(ellipse_x, ellipse_y, color='tab:purple',alpha=0.1)

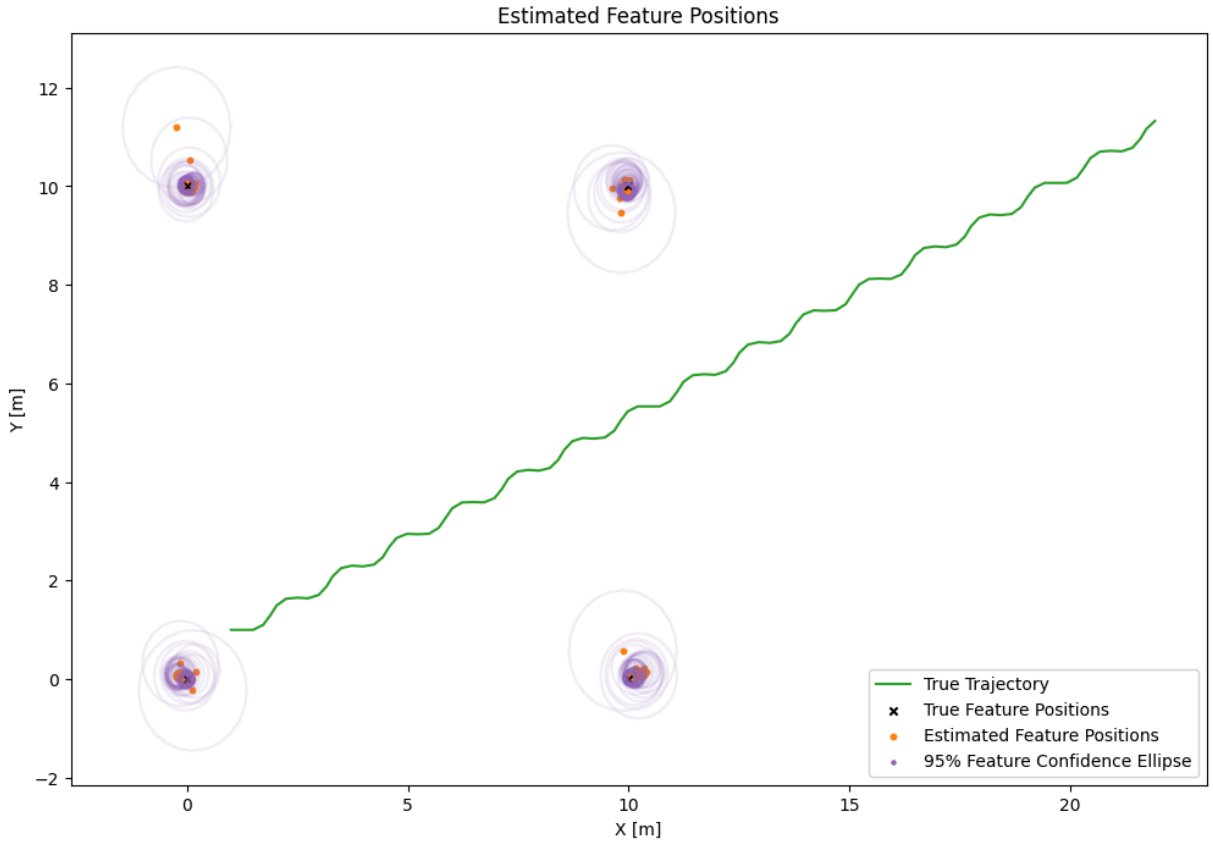



plt.scatter(mu_arr[:,0], mu_arr[:,1], s=10, color="tab:orange")
plt.scatter(mu_arr[:,2], mu_arr[:,3], s=10, color="tab:orange")
plt.scatter(mu_arr[:,4], mu_arr[:,5], s=10, color="tab:orange")
plt.scatter(mu_arr[:,6], mu_arr[:,7], s=10, color="tab:orange")

plt.scatter(measurement_locations[:,0], measurement_locations[:,1], marker="
plt.plot(X_arr[:,0], X_arr[:,1], color="tab:green")



plt.title("Estimated Feature Positions")
plt.xlabel("X [m]")
plt.ylabel("Y [m]")
plt.legend()
plt.show()
```



part d

In [565…
```python
def f_SLAM(state: np.ndarray, u: np.ndarray, dt: float):
    x, y, Theta = state[0:3]
    s, phi = u

    x_next = x + dt*s*cos(Theta)
    y_next = y + dt*s*sin(Theta)
    Theta_next = Theta + dt*phi

    state_next = state.copy()
    state_next[0:3] = [x_next, y_next, Theta_next]

    return state_next

def g_SLAM(state: np.ndarray):
    """4 Landmarks"""
    P = state[0:2]
    Theta = state[2]
    positions = state[3:11].reshape((4,2))

    A = np.array([
        [cos(Theta), sin(Theta)],
        [-sin(Theta), cos(Theta)]])

    measurements = [A @ (position - P)
        for position in positions]

    return np.array(measurements).reshape((8))
```

In [566…
```python
measurement_locations = np.array([
    [0, 0], [10, 0], [10, 10], [0, 10]
])
```

In [567…
```python
def get_A_SLAM(state: np.ndarray, u: np.ndarray, dt: float):
    """4 Landmarks"""
    _, _, Theta = state[0:3]
    s, _ = u


    A = np.eye(11)
    A[0:3, 0:3] = np.array([
        [1, 0, -dt*s*sin(Theta)],
        [0, 1,  dt*s*cos(Theta)],
        [0, 0,        1        ]
    ])
    return A

def get_C_SLAM(state: np.ndarray):
    """4 Landmarks"""
    x, y, Theta = state[0:3]
    positions = state[3:11].reshape((4,2))
```

```
    c = cos(Theta)
    s = sin(Theta)

    C_robot = []
    for pos in positions:
        row = np.array([
            [-c, -s, -s*(pos[0] - x) + c*(pos[1] - y)],
            [s, -c, -c*(pos[0] - x) - s*(pos[1] - y)]
        ])

        C_robot.append(row)
    C_robot = np.vstack(C_robot)

    R = np.array([
        [c, s],
        [-s, c]]
    )

    n_landmarks = 4
    C_landmarks = np.zeros((2*n_landmarks,2*n_landmarks))

    for i in range(0, 2*n_landmarks, 2):

        C_landmarks[i:i+2, i:i+2] = R

    C = np.hstack((C_robot, C_landmarks))

    return C


X_test = np.array([10,30,40])
measurement_locations = np.array([
    [0, 0], [10, 0], [10, 10], [0, 10]
])

X_large = np.hstack((X_test, measurement_locations.reshape(8)))

print(get_C_SLAM(X_large).shape)
```

```
(8, 11)
```

In [568…

```
dt = 0.5
TIMESTEPS = 100
measurement_locations = np.array([
    [0, 0], [10, 0], [10, 10], [0, 10]
])
SEED = 273
```

In [569…

```
Q_SLAM = np.zeros((11,11))
Q_SLAM[0:3, 0:3] = np.eye(3) * 0.1 * dt
R_SLAM = np.eye(8) * 0.25 # for 8 measurements

P0 = [1,1,0]
truth0 = np.hstack((P0, measurement_locations.reshape(8)))
```

In [570…
```
mu_pos0 = [1,1,0]
mu_loc_0 = np.array([
    [0,0],
    [0,0],
    [0,0],
    [0,0]
])
mu0 = np.hstack((mu_pos0, mu_loc_0.reshape(8)))
Sigma0 = np.eye(11)
Sigma0[3:11, 3:11] *= 1e5
```

In [571…
```
np.random.seed(SEED)
truth_arr = np.zeros((TIMESTEPS+1, 11))
meas_arr = np.zeros((TIMESTEPS+1, 8))

mu_arr = np.zeros((TIMESTEPS+1, 11))
Sigma_arr = np.zeros((TIMESTEPS+1, 11, 11))

truth_arr[0] = truth0
meas_arr[0] = g_SLAM(truth0) + noise(R_SLAM)


mu_arr[0] = mu0
Sigma_arr[0] = Sigma0

n_landmarks = 4

for i in range(TIMESTEPS):
    t = i * dt

    # Input
    u = get_u(t)

    # Truth
    truth_arr[i+1] = f_SLAM(truth_arr[i], u, dt) + noise(Q_SLAM)
    meas_arr[i] = g_SLAM(truth_arr[i]) + noise(R_SLAM)

    # EKF Predict
    mu_pred = f_SLAM(mu_arr[i], u, dt)
    A = get_A_SLAM(mu_arr[i], u, dt)
    Sigma_pred = A @ Sigma_arr[i] @ A.T + Q_SLAM

    # EKF Update
    C = get_C_SLAM(mu_pred)

    K = Sigma_pred @ C.T @ inv(C @ Sigma_pred @ C.T + R_SLAM)
    mu_arr[i+1] = mu_pred + K @ (meas_arr[i] - g_SLAM(mu_pred))

    Sigma_arr[i+1] = Sigma_pred - K @ C @ Sigma_pred

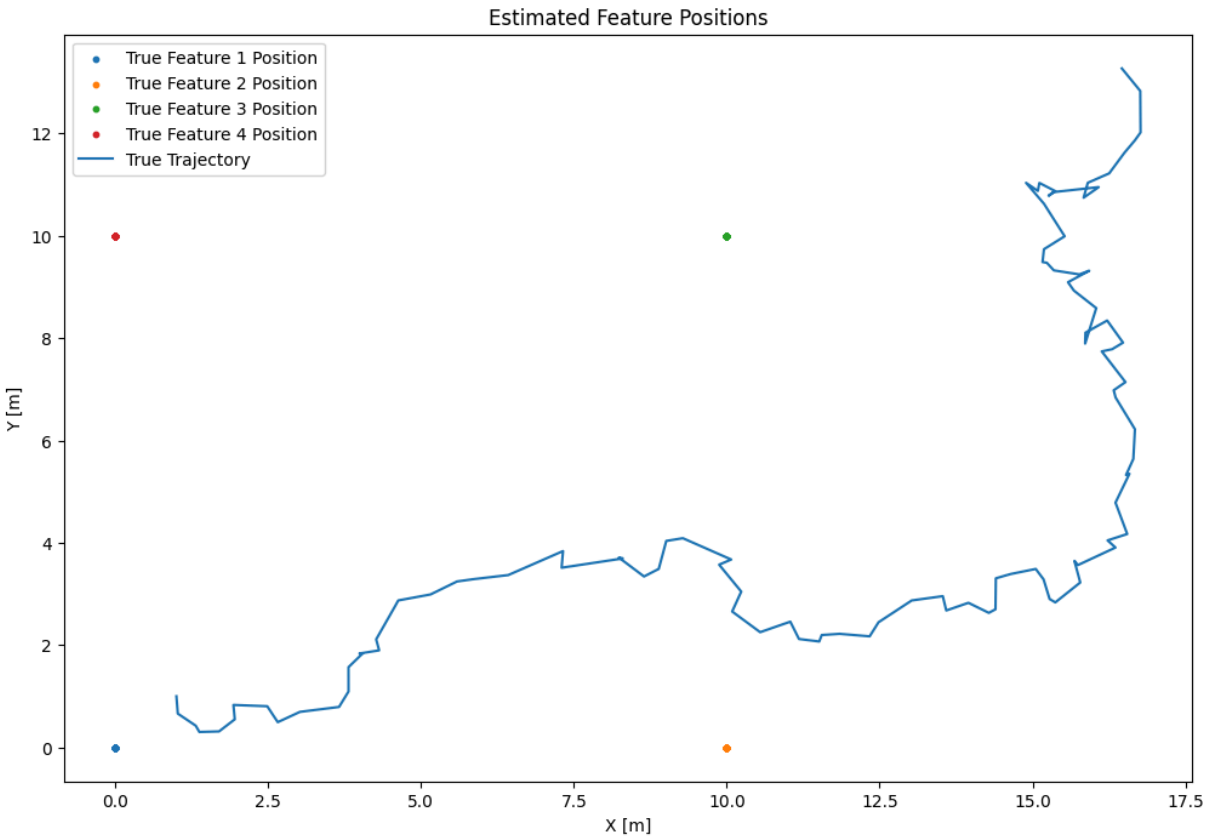
#     C_F = get_C_3_map(mu_robot_arr[i], n_landmarks)
#     features = Sigma_features_predict @ C_F.T @ inv(C_F @ Sigma_features_p
#     mu_features_arr[i+1] = mu_features_pred + features @ (meas_Y_arr[i] -
#     Sigma_features_arr[i+1] = Sigma_features_predict - features @ C_F @ Si
```

In [572…

```
plt.figure(figsize=(12,8))
plt.scatter(truth_arr[:,3], truth_arr[:,4], label="True Feature 1 Position",
plt.scatter(truth_arr[:,5], truth_arr[:,6], label="True Feature 2 Position",
plt.scatter(truth_arr[:,7], truth_arr[:,8], label="True Feature 3 Position",
plt.scatter(truth_arr[:,9], truth_arr[:,10], label="True Feature 4 Position"

plt.plot(truth_arr[:,0], truth_arr[:,1], label="True Trajectory")


plt.title("Estimated Feature Positions")
plt.xlabel("X [m]")
plt.ylabel("Y [m]")
plt.legend()
plt.show()
```



In [573…

```
plt.figure(figsize=(12,8))


plt.plot([], [], color="tab:green", label="True Trajectory")[0]
plt.plot([], [], label="Estimated Trajectory", color="tab:red")[0]
plt.scatter([], [], label="True Feature Positions", s=20, color="k", marker=
plt.scatter([], [], label="Estimated Feature Positions", s=10, color="tab:or
plt.scatter([], [], color='tab:red', zorder=2, s=5, label="95% Trajectory Co
plt.scatter([], [], color='tab:purple', s=5, label=r"95% Feature Confidence


for i, (mu, sigma) in enumerate(zip(mu_arr[1:], Sigma_arr[1:])):
    position = mu[0:2]
    position_sigma = sigma[0:2, 0:2]
    factor = 1
```

```
    ellipse_x, ellipse_y = get_ellipse_points(position, position_sigma)

    plt.plot(ellipse_x, ellipse_y, color='tab:red', zorder=2, alpha=0.1)

    for i in range(3, 11, 2):
        position = mu[i:i+2]
        position_sigma = sigma[i:i+2, i:i+2]
        factor = 1
        ellipse_x, ellipse_y = get_ellipse_points(position, position_sigma,
        plt.plot(ellipse_x, ellipse_y, color='tab:purple', linewidth=0.2, al

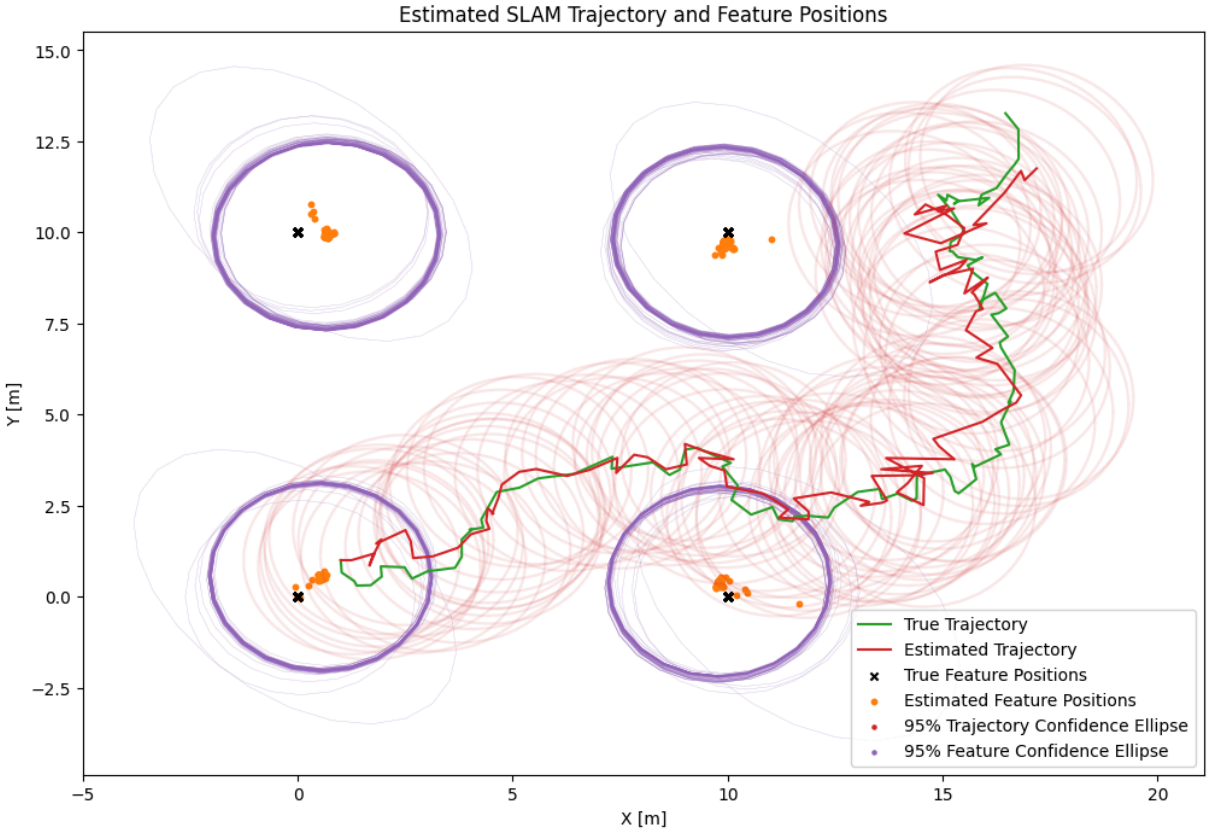


plt.plot(truth_arr[:,0], truth_arr[:,1], color="tab:green")
plt.plot(mu_arr[:,0], mu_arr[:,1], color="tab:red")

plt.scatter(mu_arr[:,3], mu_arr[:,4], s=10, color="tab:orange")
plt.scatter(mu_arr[:,5], mu_arr[:,6], s=10, color="tab:orange")
plt.scatter(mu_arr[:,7], mu_arr[:,8], s=10, color="tab:orange")
plt.scatter(mu_arr[:,9], mu_arr[:,10], s=10, color="tab:orange")

plt.scatter(truth_arr[:,3], truth_arr[:,4], s=20, color="k", marker="x")
plt.scatter(truth_arr[:,5], truth_arr[:,6], s=20, color="k", marker="x")
plt.scatter(truth_arr[:,7], truth_arr[:,8], s=20, color="k", marker="x")
plt.scatter(truth_arr[:,9], truth_arr[:,10], s=20, color="k", marker="x")

plt.title("Estimated SLAM Trajectory and Feature Positions")
plt.xlabel("X [m]")
plt.ylabel("Y [m]")
plt.legend()
plt.show()
```


Estimated SLAM Trajectory and Feature Positions
15.0
12.5
10.0
7.5
5.0
2.5
0.0
−2.5
Y [m]
−5
0
5
10
15
20
X [m]
True Trajectory
Estimated Trajectory
True Feature Positions
Estimated Feature Positions
95% Trajectory Confidence Ellipse
95% Feature Confidence Ellipse